

LAB-8

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVES

- 1) To understand Syntax Directed Translation.
- 2) To design Context Free Grammar (CFG) for arithmetic expressions.
- 3) To attach semantic rules to grammar productions.
- 4) To implement SDT using Lex and Yacc.
- 5) To generate intermediate outputs during parsing.
- 6) To perform semantic analysis like type checking.
- 7) To translate infix expressions to postfix form.

PROBLEM STATEMENT

- 1) Design an SDT scheme to evaluate arithmetic expressions with operators +, -, *, / and parentheses while respecting precedence and associativity.
- 2) Develop an SDT scheme to check types for assignment and arithmetic expressions that involve int and float data types. Find type mismatch errors using a symbol table.
- 3) Design an SDT scheme to convert infix arithmetic expressions into postfix notation for execution in a stack-based virtual machine.

AIM

To implement Syntax Directed Translation (SDT) using Lex and Yacc for:

1. Evaluating arithmetic expressions.
2. Performing type checking in assignment statements.
3. Converting infix expressions into postfix notation.

PROCEDURE

- 1) Provide a CFG to represent arithmetic expressions.
- 2) Provide appropriate semantic rules for:
- 3) Evaluation of Expressions
- 4) Type Checking
- 5) Infix/Postfix Conversion
- 6) Write a lexical analyzer using Lex to identify tokens.
- 7) Write a parser using Yacc with semantic actions.
- 8) Compile programs as follows:
- 9) `yacc -d <FileName>.y`
- 10) `lex <FileName>.lex`
- 11) `gcc lex.yy.c y.tab.c -o <Output>`
- 12) `<Execute Output>`
- 13) Run the program and test it with sample data.

Syntax Directed Translation (SDT)

Scenario:

A scientific calculator application accepts arithmetic expressions from users. The calculator must compute results by respecting operator precedence and associativity rules. To improve efficiency, the compiler designer decides to use **Syntax Directed Translation (SDT)** to evaluate expressions during parsing.

Task:

Design an SDT scheme for evaluating arithmetic expressions containing operators $+$, $-$, $*$, $/$, and parentheses.

Requirements:

1. Write a Context Free Grammar (CFG) for arithmetic expressions.
2. Attach semantic rules to compute expression values.
3. Implement the SDT using **Lex and Yacc**.
4. Test your program with expressions such as:
5. $5 + 3 * 2$
6. $(7 + 4) * 6$
7. $10 / 2 + 8$
8. Display the computed result after parsing.

CODE:

LEX

```
%{  
  
#include "y.tab.h"  
  
#include <stdlib.h>  
  
%}  
  
%%  
  
[0-9]+    { yyval = atoi(yytext); return NUM; }  
  
[ \t]     ;  
  
\n        return '\n';  
  
.         return yytext[0];  
  
%%  
  
int yywrap() {  
    return 1;  
}
```

YACC

```
%{  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
int yylex();  
void yyerror(char *s);  
  
%}
```

%token NUM

%left '+' '-'

%left '*' '/'

%%

input:

input line

|

;

line:

E '\n' { printf("Result = %d\n", \$1); }

;

E:

E '+' T { \$\$ = \$1 + \$3; }

| E '-' T { \$\$ = \$1 - \$3; }

| T { \$\$ = \$1; }

;

T:

T '*' F { \$\$ = \$1 * \$3; }

| T '/' F { \$\$ = \$1 / \$3; }

| F { \$\$ = \$1; }

;

F:

'(E '){ \$\$ = \$2; }

| NUM { \$\$ = \$1; }

;

%%

```
void yyerror(char *s) {  
    printf("Error: %s\n", s);  
}
```

```
int main() {  
    printf("Enter Expression:\n");  
    yyparse();  
    return 0;  
}
```

OUTPUT:

```
student@student-VirtualBox:~$ nano calc.l
student@student-VirtualBox:~$ nano calc.y
student@student-VirtualBox:~$ yacc -d calc.y
student@student-VirtualBox:~$ lex calc.l
student@student-VirtualBox:~$ gcc lex.yy.c y.tab.c -o calc
student@student-VirtualBox:~$ ./calc
Enter Expression:
5 + 3 * 2
Result = 11
(7 + 4) * 6
Result = 66
10 / 2 + 8
Result = 13
student@student-VirtualBox:~$
```

Question 2 – Type Checking using SDT

Scenario:

A programming language compiler must detect type mismatch errors during compilation. The compiler designer decides to integrate **type checking using Syntax Directed Translation** to verify variable compatibility in expressions.

Task:

Develop an SDT scheme to perform type checking for assignment and arithmetic expressions.

Requirements:

1. Consider data types: int and float.
2. Write grammar rules for assignment statements.
3. Attach semantic rules to:
 - Store variable types in symbol table
 - Detect incompatible operations
4. Implement using **Lex and Yacc**.
5. Validate using inputs:
6. int a;
7. float b;
8. a = b + 5;
9. b = a + 2.5;

CODE

LEX

```
%{  
  
#include "y.tab.h"  
  
#include <stdlib.h>  
  
#include <string.h>  
  
%}  
  
%%  
  
"int"      { return INT; }  
  
"float"    { return FLOAT; }  
  
[0-9]+ "." [0-9]+ {  
    yylval.fval = atof(yytext);  
    return FNUM;  
}  
  
[0-9]+     {  
    yylval.ival = atoi(yytext);  
    return NUM;  
}
```

```

    }

[a-zA-Z_][a-zA-Z0-9_]* {
    strcpy(yylval.name, yytext);
    return ID;
}

```

```

[ \t]      ;
\n         ;
.          { return yytext[0]; }
%%

```

```

int yywrap() {
    return 1;
}

```

YACC

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(const char *s);

/* ----- Symbol Table ----- */

```

```

struct symtab {
    char name[20];
    int type; /* 1 = int, 2 = float */
} table[100];

int count = 0;

/* Lookup function */
int lookup(char *name) {
    for(int i = 0; i < count; i++) {
        if(strcmp(table[i].name, name) == 0)
            return table[i].type;
    }
    return -1;
}

/* Insert function */
void insert(char *name, int type) {
    strcpy(table[count].name, name);
    table[count].type = type;
    count++;
}

%}

/* ----- UNION ----- */

```

```

%union {
    int ival;
    float fval;
    char name[20];
    int type;
}

/* ----- TOKENS ----- */

%token INT FLOAT
%token <ival> NUM
%token <fval> FNUM
%token <name> ID

/* ----- NON-TERMINAL TYPES ----- */

%type <type> TYPE E T

%%

program:
    program statement
    |
    ;

statement:
    TYPE ID ';'

```

```

    {
        insert($2, $1);
        printf("Declared successfully\n");
    }

| ID '=' E ';'
{
    int t = lookup($1);

    if(t == -1)
        printf("Error: Variable not declared\n");
    else if(t != $3)
        printf("Type Mismatch Error\n");
    else
        printf("Assignment Correct\n");
}

;

/* ----- TYPE ----- */

TYPE:

    INT { $$ = 1; }
    | FLOAT { $$ = 2; }

;

/* ----- EXPRESSIONS ----- */

```

E:

```
E '+' T
{
    if($1 == 2 || $3 == 2)
        $$ = 2;
    else
        $$ = 1;
}
| T
{
    $$ = $1;
}
;
```

T:

```
NUM { $$ = 1; }
| FNUM { $$ = 2; }
| ID
{
    int t = lookup($1);
    if(t == -1)
        printf("Error: Variable not declared\n");
    $$ = t;
}
;
```

%%

```
int main() {  
    printf("Enter statements:\n");  
    yyparse();  
    return 0;  
}  
  
void yyerror(const char *s) {  
    printf("Syntax Error\n");  
}
```

OUTPUT:

```
student@student-VirtualBox:~$ nano type.l  
student@student-VirtualBox:~$ nano type.y  
student@student-VirtualBox:~$ yacc -d type.y  
student@student-VirtualBox:~$ lex type.l  
student@student-VirtualBox:~$ gcc lex.yy.c y.tab.c -o type  
student@student-VirtualBox:~$ ./type  
Enter statements:  
int a;  
float b;  
a = b + 5;  
b = a + 2.5;  
Declared successfully  
Declared successfully  
Type Mismatch Error  
Assignment Correct
```

Question 3 – Infix to Postfix Translation using SDT

Scenario:

A compiler for a stack-based virtual machine requires expressions to be converted into postfix notation before execution. The developer uses **Syntax Directed Translation** to convert infix expressions into postfix form during parsing.

Task:

Design an SDT scheme to translate infix expressions into postfix expressions.

Requirements:

1. Write CFG for arithmetic expressions.
2. Attach semantic rules to generate postfix output.
3. Implement using **Lex and Yacc**.
4. Test using inputs:
5. $A + B * C$

6. $(A + B) * C$

7. $A * B + C / D$

8. Display postfix expression.

CODE:

LEX

```
%{  
#include "y.tab.h"  
%}  
  
%%  
[a-zA-Z]    { yyval.ch = yytext[0]; return ID; }  
[ \t]       ;  
\n         return '\n';  
.  
            { return yytext[0]; }  
%%  
  
int yywrap() {  
    return 1;  
}
```

YACC

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int yylex();
```

```
void yyerror(const char *s);
```

```
%}
```

```
%union {
```

```
    char ch;
```

```
}
```

```
%token <ch> ID
```

```
%left '+' '-'
```

```
%left '*' '/'
```

```
%%
```

```
input:
```

```
    input line
```

```
    |
```

```
    ;
```

```
line:
```

```
    E '\n' { printf("\n"); }
```

;

E:

E '+' T { printf("+ "); }

| E '-' T { printf("- "); }

| T

;

T:

T '*' F { printf("* "); }

| T '/' F { printf("/ "); }

| F

;

F:

(' E ')

| ID { printf("%c ", \$1); }

;

%%

```
int main() {  
    printf("Enter Infix Expression:\n");  
    yyparse();  
    return 0;  
}
```

```
}

void yyerror(const char *s) {
    printf("Syntax Error\n");
}
```

OUTPUT

```
student@student-VirtualBox:~$ nano postfix.l
student@student-VirtualBox:~$ nano postfix.Y
student@student-VirtualBox:~$ yacc -d postfix.y
student@student-VirtualBox:~$ lex postfix.l
student@student-VirtualBox:~$ gcc lex.yy.c y.tab.c -o postfix
student@student-VirtualBox:~$ ./postfix
Enter Infix Expression:
A + B * C
A B C * +
(A + B) * C
A B + C *
A * B + C / D
A B * C D / +
```

RESULT

The Lex and Yacc tools worked together properly and resulted in the successful implementation of the Syntactic Directed Translation schemes.

The evaluation of arithmetic expressions was done accurately

The detection of type mismatches when doing assignments was identified.

Conversion of infix notation to postfix was done accurately.

Therefore all of the expected outputs were successfully produced.